

# Claude Code 快速实践

前面我们完成了从"安装部署"到"六大系统游览"的认知建设。但 Claude Code 的真正威力，只有在**完整项目的开发过程中**才能充分体现。本节将以一个**Web 端大模型聊天机器人（ChatBot）**为目标，从前端设计到后端开发，带你体验两种截然不同的开发模式——"一句话全自动生成"与"多 Agent 流水线协作"。

这不是一个简单的 Hello World 练习。完成本节后，你将拥有一个可运行的 ChatBot 应用，更重要的是，你将掌握 Claude Code 在项目开发中的核心工作方法论。

## 1、需求分析与 HTML 原型设计

任何项目的第一步都是**明确需求**。我们的目标是开发一个 Web 端的大模型聊天机器人，具体需求如下：

| 项目开发需求 (CHALKBOARD VERSION) |                                           |
|-----------------------------|-------------------------------------------|
| 维度                          | 要求                                        |
| 界面布局                        | 左侧历史会话列表 + 右侧聊天主区域，深色主题                   |
| 消息展示                        | 用户消息靠右（浅色气泡）、AI 消息靠左（深色气泡），支持 Markdown 渲染 |
| 输入交互                        | 底部输入框 + 发送按钮，支持 Enter 发送 / Shift+Enter 换行 |
| 模拟功能                        | 先实现前端原型，AI 回复暂用模拟数据（打字机逐字效果）              |
| 技术约束                        | 单 HTML 文件，内联 CSS + JS，无外部依赖，浏览器直接打开即可运行   |

**为什么从单 HTML 文件开始？** 因为这是验证前端设计的最快路径——无需启动服务器、无需安装依赖，双击打开就能看到效果。后续再拆分为前后端分离的完整应用。

这次我使用的是 VS Code 工具集成 Cluade Code 的方式演示。

### 1.1 CLI 实操：让 Claude Code 生成初版页面

创建项目目录并启动 Claude Code：

代码块

```
1  # 创建项目目录
2  mkdir ~/chatbot-project
3  #进入项目
4  cd ~/chatbot-project
5  # 进入 cluade模式
```

```
PS D:\code\claude-lab\chatbot-project> claude
Claude Code v2.1.81

Welcome back!



Sonnet 4.6 · API Usage Billing
D:\code\claude-lab\chatbot-project

Tips for getting started
Ask Claude to create a new app or clone a repository

Recent activity
No recent activity

↑ Opus now defaults to 1M context · 5x more room, same pricing

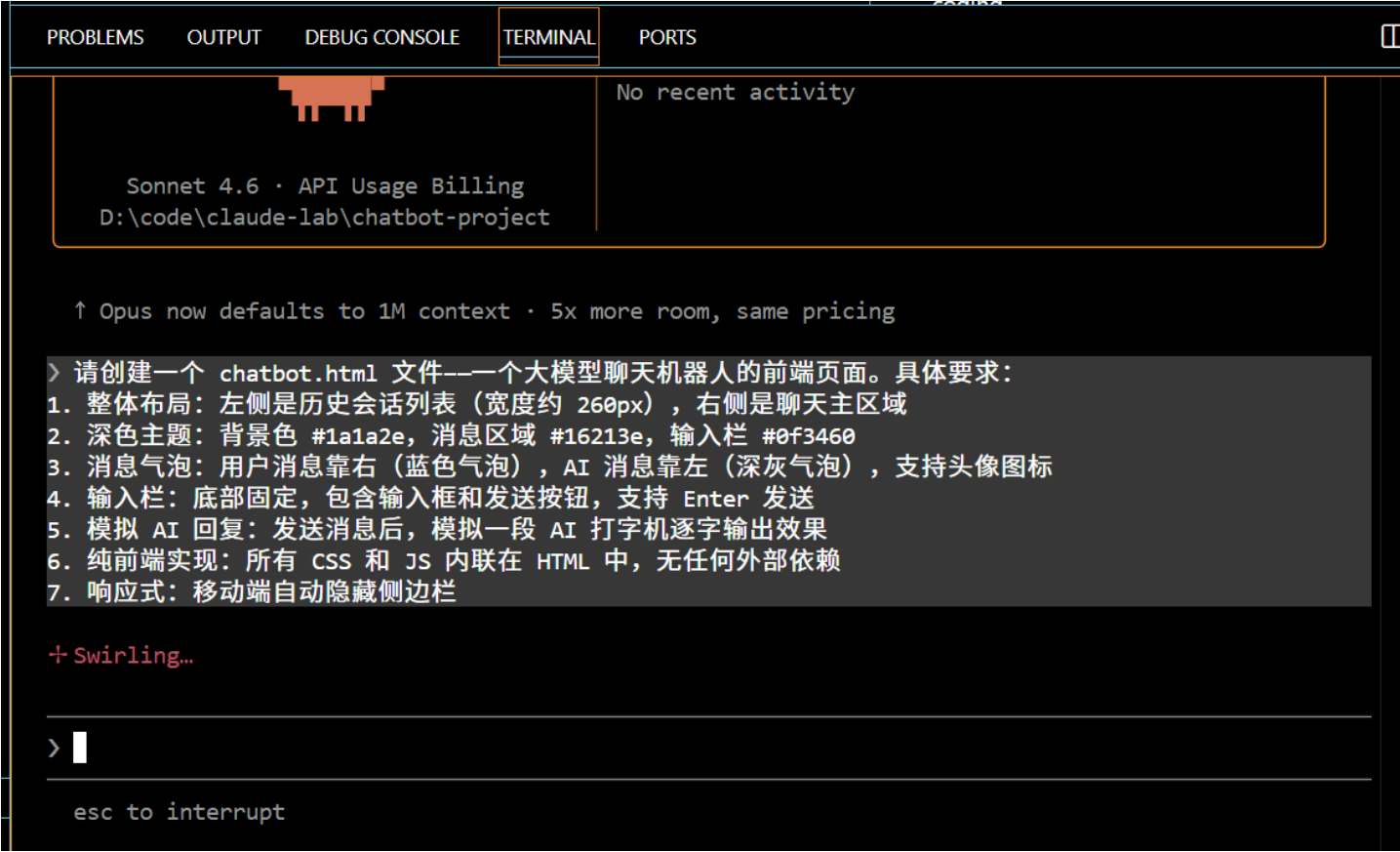
> Try "write a test for <filepath>"

? for shortcuts medium · /effort
```

在对话中输入以下需求（注意：Prompt 的质量直接决定输出质量，所以我们把需求描述得足够具体）：

#### 代码块

- 1 请在当前目录创建一个 chatbot.html 文件——一个大模型聊天机器人的前端页面。具体要求：
- 2 1. 整体布局：左侧是历史会话列表（宽度约 260px），右侧是聊天主区域
- 3 2. 深色主题：背景色 #1a1a2e，消息区域 #16213e，输入栏 #0f3460
- 4 3. 消息气泡：用户消息靠右（蓝色气泡），AI 消息靠左（深灰气泡），支持头像图标
- 5 4. 输入栏：底部固定，包含输入框和发送按钮，支持 Enter 发送
- 6 5. 模拟 AI 回复：发送消息后，模拟一段 AI 打字机逐字输出效果
- 7 6. 纯前端实现：所有 CSS 和 JS 内联在 HTML 中，无任何外部依赖
- 8 7. 响应式：移动端自动隐藏侧边栏



几秒钟后，双击打开 chatbot.html ——一个完整的深色主题聊天界面已经呈现在浏览器中。试着输入一条消息并发送，你会看到模拟的 AI 回复以打字机效果逐字出现。



## 2、前端优化（一）： /frontend-design 插件

Claude Code 生成的前端页面，在功能层面通常没有问题，但在设计品味上往往还是差点意思。这并非模型能力不足，而是一个统计学现象——大语言模型倾向于生成高概率的通用方案（Inter 字体、紫色渐变、圆角卡片），因为这些组合在训练数据中出现频率最高。Anthropic 官方为此专门开发了一个插件：**frontend-design**。

## 2.1 /frontend-design 是什么？

| CLAUDE DESIGN SKILL - 黑板报 |                                                                                                                           |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------|
| 属性 (PROPERTY)             | 说明 (DESCRIPTION)                                                                                                          |
| 开发者                       | Anthropic 官方团队                                                                                                            |
| 类型                        |  Claude Code Plugin (包含一个 Skill)         |
| 安装量                       |  24万+ (截至 2026 年 3 月)                    |
| 核心作用                      | 注入专业设计指导, 让 Claude 在前端任务中自动产出更有辨识度的界面  |
| 触发方式                      | 自动激活——安装后无需手动调用, Claude 检测到前端任务时自动应用   |

它的工作原理是：在 Claude 的系统提示中注入约 400 token 的**专业设计准则**，覆盖排版、配色、动效、背景四个维度，引导模型跳出"安全方案"的舒适区。

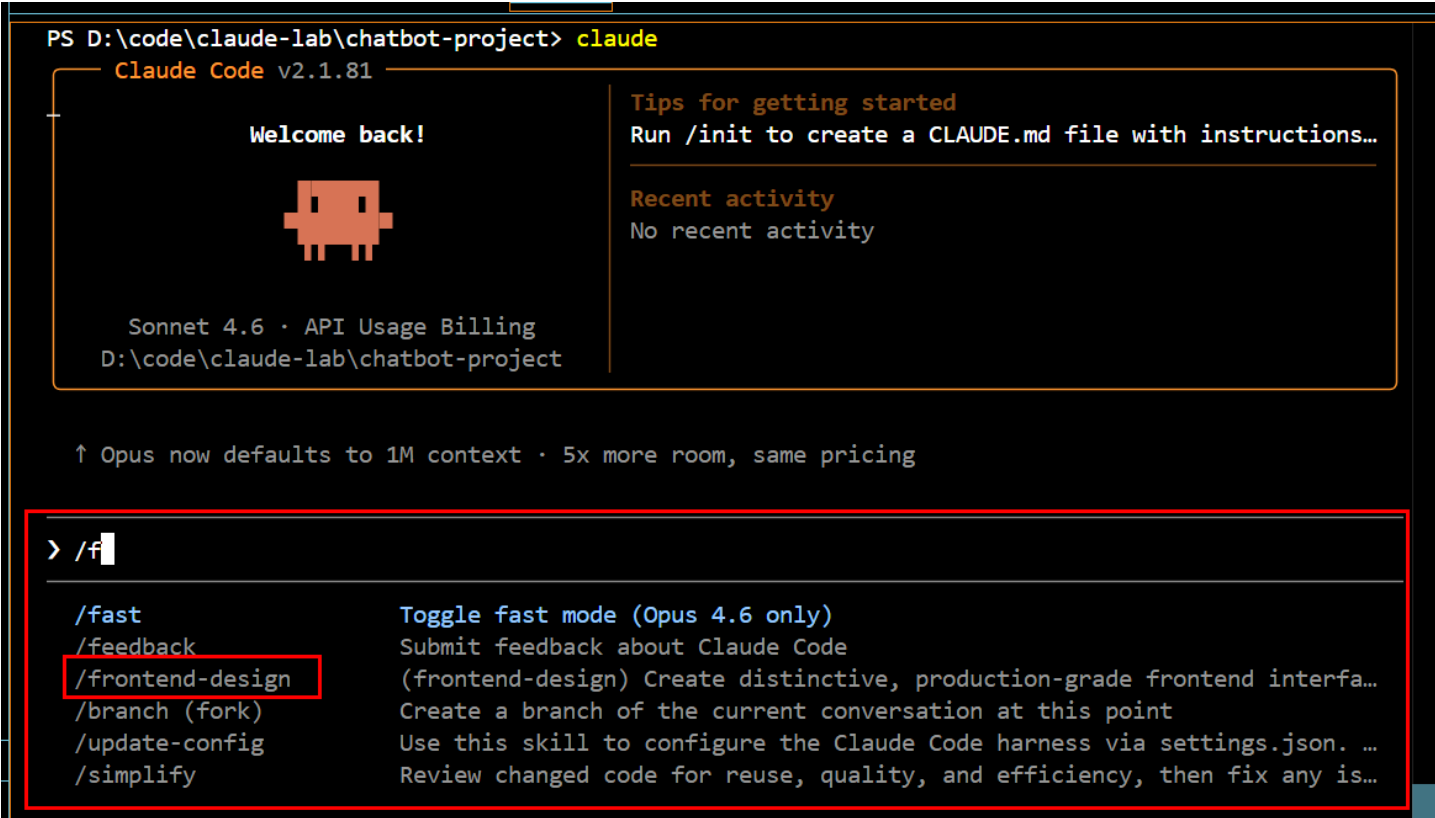
## 2.2 安装方法

在 Claude Code 中执行一条命令即可安装：

代码块

```
1 帮我安装frontend-design插件！
```

安装完成后，Claude Code 会提示 Successfully installed frontend-design 。无需重启，下一次涉及前端的对话中它就会自动生效。



**查看完整 Prompt:** 如果你好奇这个插件到底给 Claude 注入了什么指令，可以在 GitHub 上查看它的完整 SKILL.md 文件：

|                                                                         |  |                 |               |   |
|-------------------------------------------------------------------------|--|-----------------|---------------|---|
| nd-design > 15268f03d2f5 > skills > frontend-design                     |  |                 |               |   |
| <div><div>🗑</div><div>↕ 排序 ▾</div><div>☰ 查看 ▾</div><div>...</div></div> |  |                 |               |   |
| 名称                                                                      |  | 修改日期            | 类型            | 大 |
| SKILL.md                                                                |  | 2026/3/24 13:16 | Markdown File |   |

```
name: frontend-design
description: Create distinctive, production-grade
frontend interfaces with high design quality. Use this
skill when the user asks to build web components, pages,
or applications. Generates creative, polished code that
avoids generic AI aesthetics.
license: Complete terms in LICENSE.txt
```

This skill guides creation of distinctive, production-grade frontend interfaces that avoid generic "AI slop" aesthetics. Implement real working code with exceptional attention to aesthetic details and creative choices.

The user provides frontend requirements: a component, page, application, or interface to build. They may include context about the purpose, audience, or technical constraints.

## Design Thinking

Before coding, understand the context and commit to a BOLD aesthetic direction:

- **Purpose:** What problem does this interface solve? Who

2.3 优化维度一览

| 维度                                                                                   | 默认行为 (无插件)                                                                               | 优化后 (有插件)                                                                                                                   |
|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
|  排版 | Arial / Inter / Roboto 等通用字体                                                             | 有辨识度的字体搭配 (如 Playfair Display + 精炼正文字体)  |
|  配色 | 平均分布的“安全色”                                                                               | 主色 + 锐利强调色, CSS 变量保持一致性                  |
|  动效 | 无或极简过渡  | 编排入场动画 (staggered reveals)、滚动触发、悬停状态                                                                                        |
|  背景 | 纯色或简单渐变                                                                                  | 渐变网格、噪点纹理、几何图案、分层透明                      |

网页设计与开发：从默认到优化

2.4 实操：优化 ChatBot 页面

安装插件后，让 Claude Code 重新审视并优化我们的 ChatBot 页面：

代码块

1

/frontend-design 请对 chatbot.html 进行一次前端设计升级——不改变功能逻辑，

2

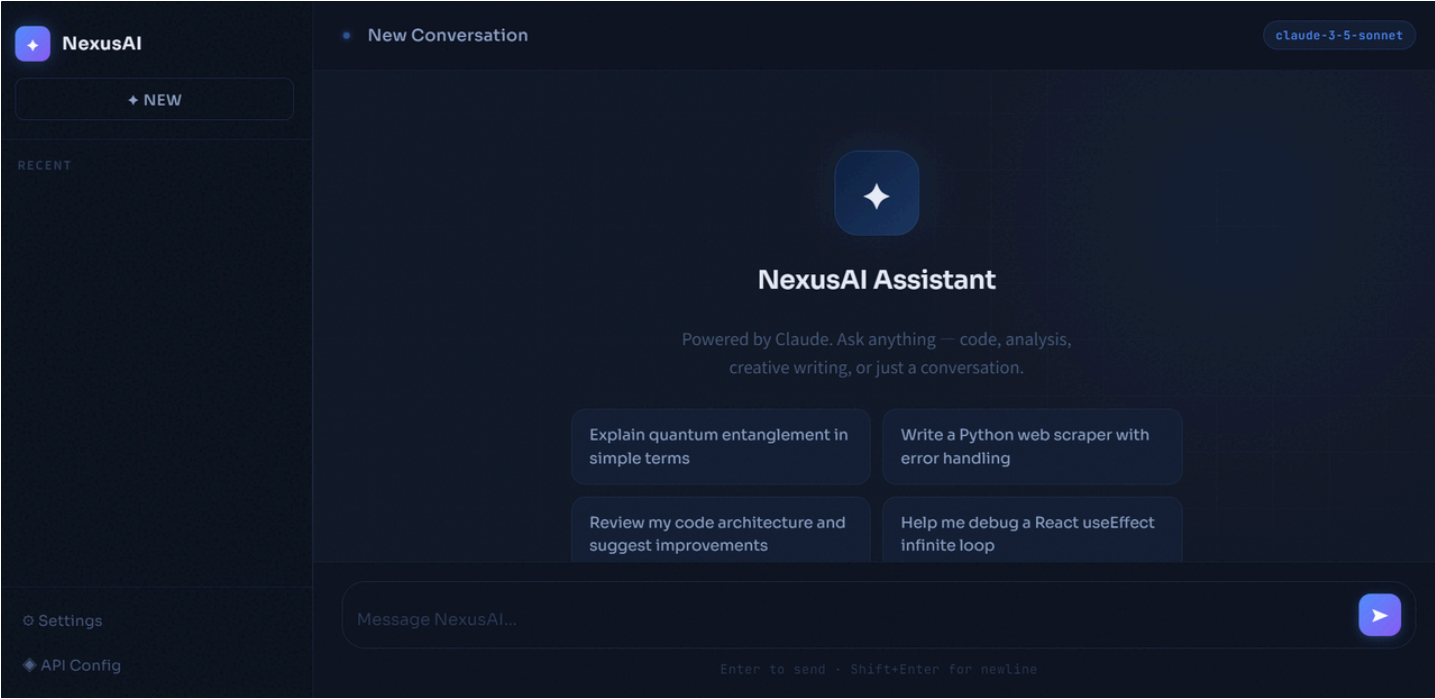
重点提升视觉品质：优化排版层级、增加微交互动效、提升色彩层次感、让界面看起来更像一个精心设计的产品而非 AI 生成的原型。

由于 frontend-design 插件已自动激活，Claude 在处理这个请求时会参考专业设计准则，产出的结果会与初版有**明显的视觉差异**——更精致的字体排版、更有层次的色彩方案、精心编排的过渡动画。

执行完以后的效果：



/frontend-design 请对 chatbot.html 进行一次前端设计升级——不改变功能逻辑，左边的排版要整齐一些。



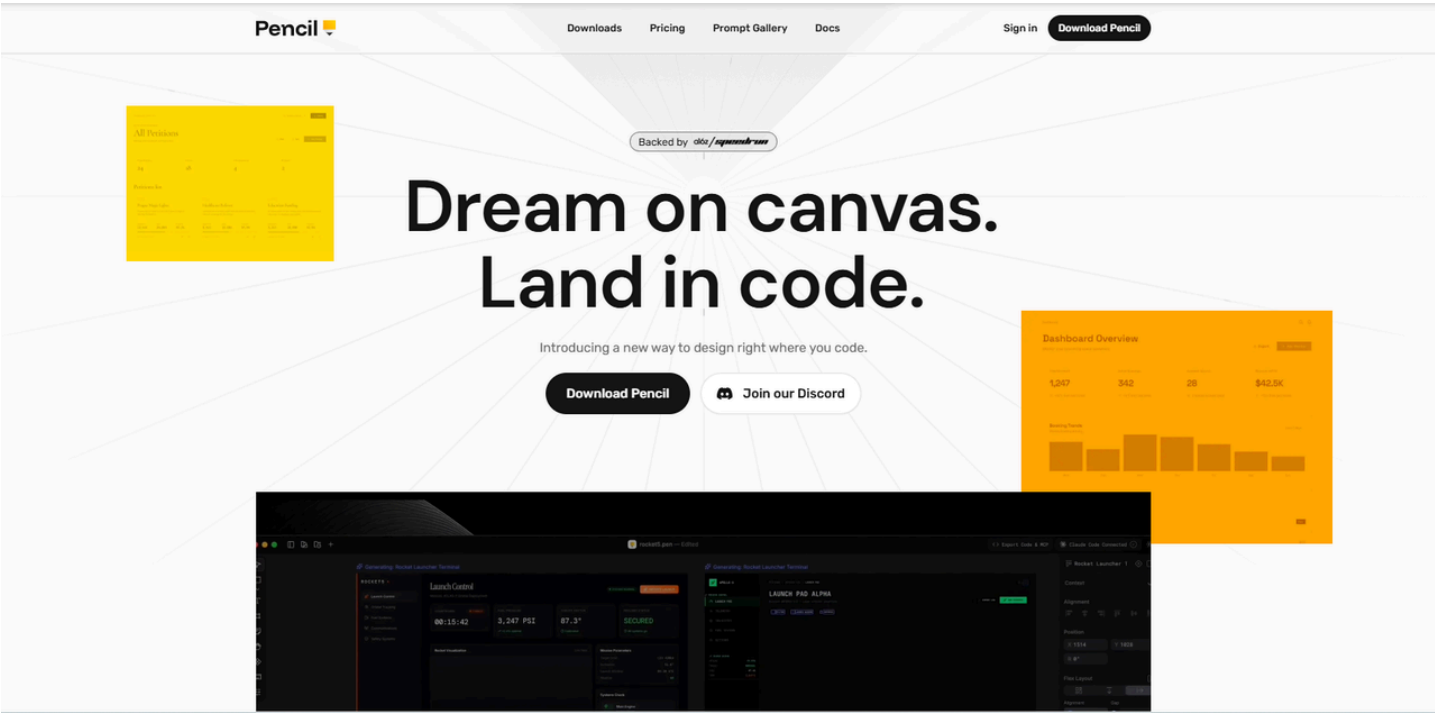
### 3、前端优化（二）：Pencil 设计工具

/frontend-design 插件通过文字指令提升设计品质，但有些视觉调整——比如精确的间距、色块比例、组件布局——用语言描述效率很低。这时候需要一个**可视化设计工具**。Pencil 正是为此而生。

#### 3.1 Pencil 是什么？

| PENCIL TOOL: CHALKBOARD OVERVIEW |                                          |
|----------------------------------|------------------------------------------|
| 属性                               | 说明                                       |
| 产品定位                             | AI 原生设计工具（类 Figma 的无限画布）                 |
| 核心能力                             | 通过 MCP 协议与 Claude Code 深度集成，AI 可直接操控设计画布 |
| 文件格式                             | .pen 格式（加密，只能通过 Pencil 工具读写）             |
| 官网                               | pencil.dev ←                             |

Pencil 与 Claude Code 的关系是：**Pencil 负责视觉设计，Claude Code 负责代码实现，MCP 协议在两者之间架起桥梁**。安装 Pencil 后，Claude Code 可以直接在对话中创建设计元素、调整布局、生成截图预览，甚至从设计稿直接生成代码。



### 3.2 Windows 安装指南

Pencil 在 Windows 上有两种安装方式：

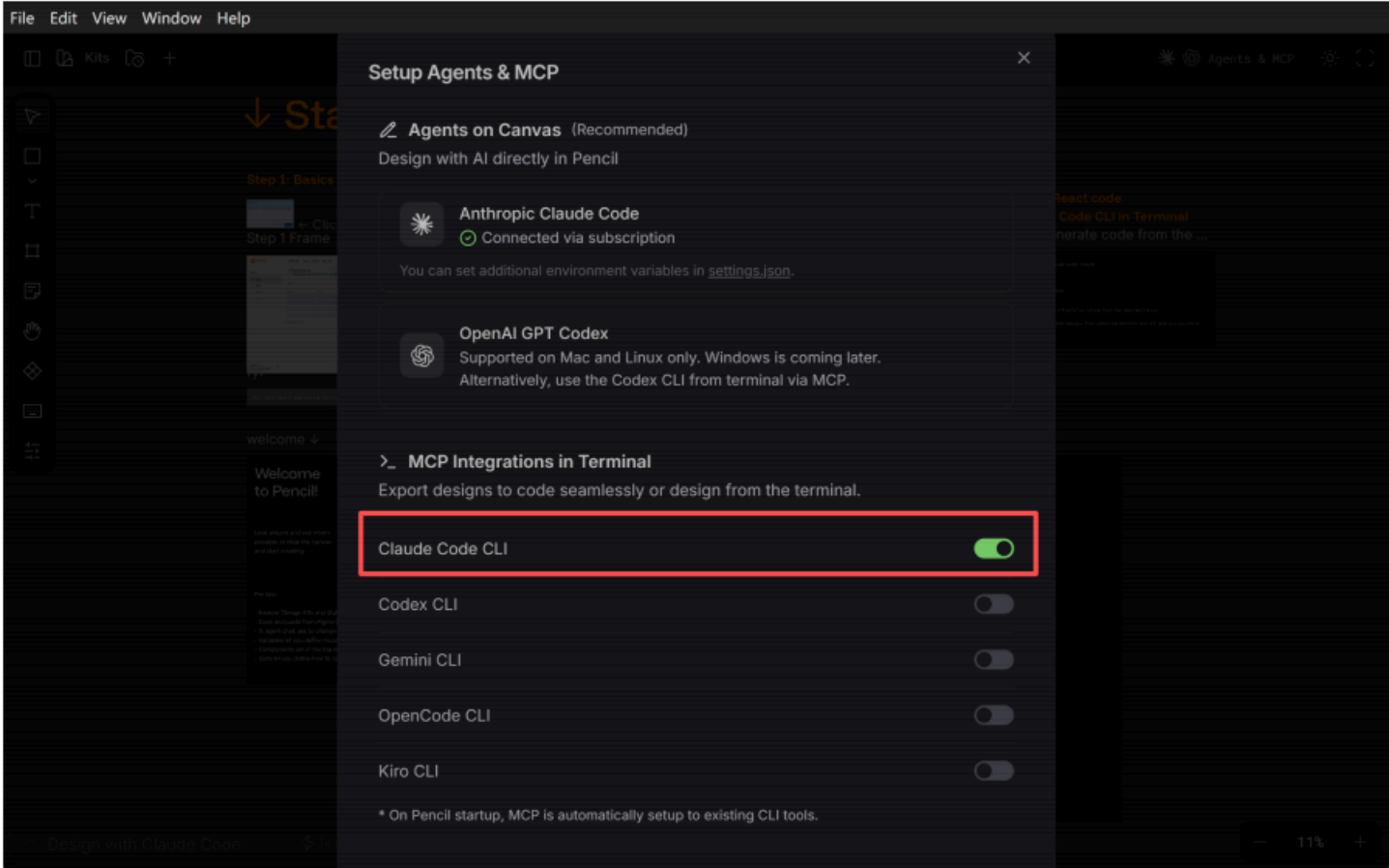
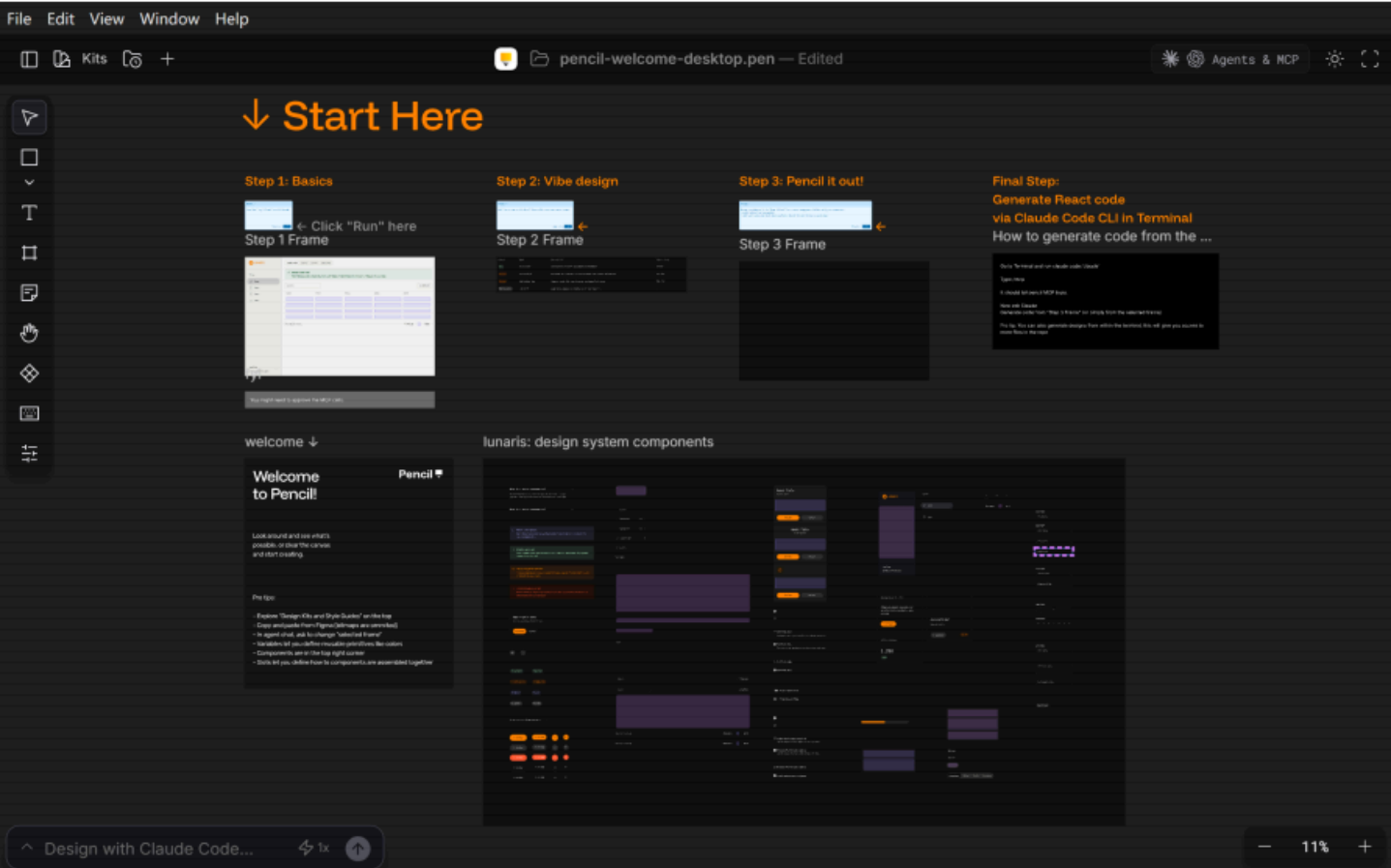
#### 方式一：VS Code 扩展（推荐）

- 1. 打开 VS Code → 扩展面板（Ctrl+Shift+X）
- 2. 搜索 "Pencil"，确认标识符为 highagency.pencildev
- 3. 点击安装
- 4. 用邮箱完成激活

#### 方式二：桌面应用

- 1. 访问 pencil.dev/downloads
- 2. 下载 Windows .exe 安装包

3. 运行安装程序，按提示完成



3.3 验证 MCP 连接

安装完成后，在 Claude Code 中验证 Pencil MCP 是否已正确连接：

## 代码块

```
1 /mcp
2 ## 然后敲确认键
```

```
claude --resume b7f49874-46a8-46e1-88ad-421d13695e0c
PS D:\code\claude-lab\chatbot-project> claude
Claude Code v2.1.81

Welcome back!



Sonnet 4.6 · API Usage Billing
D:\code\claude-lab\chatbot-project

Tips for getting started
Run /init to create a CLAUDE.md file with instructions f...

Recent activity
No recent activity

> /mcp

Manage MCP servers
1 server

User MCPs (C:\Users\ZhuanZ\claude.json)
> pencil · ✓ connected

https://code.claude.com/docs/en/mcp for help
↑↓ to navigate · Enter to confirm · Esc to cancel
```

在 MCP Server 列表中，应该看到 pencil 显示为 **connected** 状态。如果未显示，尝试重启 Claude Code。

### 3.4 Pencil 暴露的核心工具

通过 MCP 协议，Pencil 向 Claude Code 暴露了以下工具，Claude 可以在对话中直接调用：

| 开发者工具与用途 (DEVELOPER TOOLS & USES) |                                                 |                    |
|-----------------------------------|-------------------------------------------------|--------------------|
| 工具 (TOOL)                         | 功能 (FUNCTION)                                   | 典型用途 (TYPICAL USE) |
| batch_design                      | 创建/修改设计元素 (插入、复制、更新、替换、移动、删除)                   | 在画布上构建 UI 布局       |
| batch_get                         | 读取组件和节点信息                                       | 分析现有设计结构           |
| get_screenshot                    | 渲染指定节点的截图                                       | 预览设计效果             |
| get_guidelines                    | 获取设计规范 (landing-page / design-system / table 等) | 确保设计符合规范           |
| get_style_guide                   | 获取风格指南                                          | 保持视觉风格一致性          |
| snapshot_layout                   | 检查当前布局结构                                        | 排查布局问题             |

3.5 实操：用 Pencil 精调 ChatBot 设计

在 Claude Code 中输入：

代码块

1

请使用 Pencil 设计工具，为 chatbot.html 创建一份设计稿。

2

在 Pencil 画布上绘制 ChatBot 的界面布局，包含：

3

1. 整体页面框架（左侧边栏 + 右侧聊天区）

4

2. 消息气泡组件（用户/AI 两种样式）

5

3. 输入栏组件

6

使用深色主题配色方案

\* Cogitated for 53s

> 请使用 Pencil 设计工具，为 chatbot.html 创建一份设计稿。  
在 Pencil 画布上绘制 ChatBot 的界面布局，包含：  
1. 整体页面框架（左侧边栏 + 右侧聊天区）  
2. 消息气泡组件（用户/AI 两种样式）  
3. 输入栏组件  
使用深色主题配色方案

\* Swooping...

> █

esc to interrupt

Claude Code 会调用 Pencil MCP 的工具——你可以在终端中看到 batch\_design、get\_screenshot 等工具调用。设计完成后，Claude 会自动截图预览。

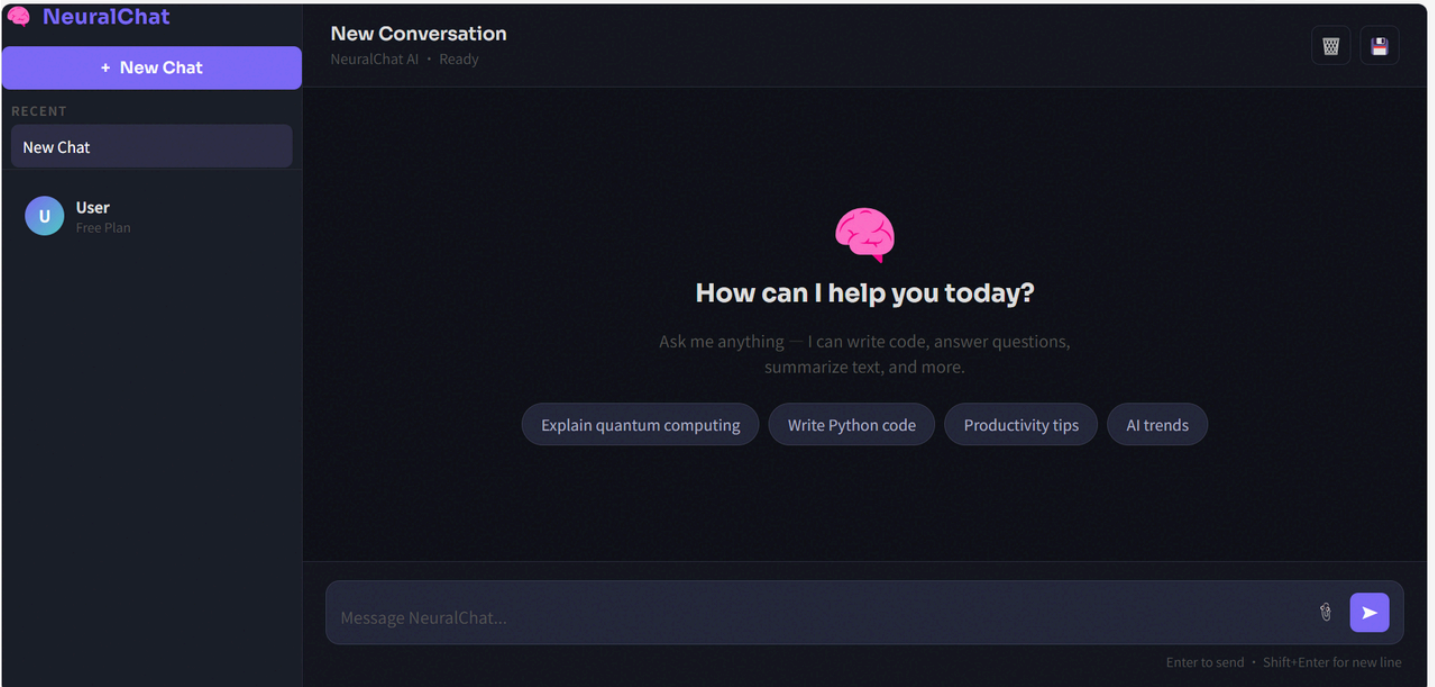


如果想从设计稿生成代码，可以继续输入：

代码块

1 请根据 Pencil 中的设计稿，更新当前目录下的chatbot.html 的样式，确保代码与设计稿一致。

如下是重新根据设计搞生成的UI：



## 4、代码开发（方案一）：一句话生成完整应用

前端设计已经满意了。现在进入真正的开发环节——把这个前端原型变成一个前后端完整的 ChatBot 应用，能够真正调用大模型 API 进行对话。

方案一是最简单直接的方式：用一句话让 Claude Code 生成全部代码

## 4.1 需求 Prompt

这是本节最关键的一步——**Prompt Engineering** 的质量直接决定生成代码的质量。以下是一个经过精心设计的需求描述：

代码块

```
1  基于当前目录中的 chatbot.html 前端设计，请帮我开发一个完整的前后端 ChatBot 应用。
2  具体要求：
3  【前端】
4  - 以 chatbot.html 的设计为基础，改为通过 fetch API 调用后端接口
5  - 发送消息时调用 POST /api/chat，传递 messages 数组（多轮对话）
6  - AI 回复支持流式输出（Server-Sent Events），逐字显示
7  【后端】
8  - Node.js + Express
9  - 提供 POST /api/chat 接口
10 - 后端通过调用DeepSeek的模型，实现流式响应，我的key是：sk-
    553c1c39b02f40c193a2b0e4b8d41b3c
11 - 维护多轮对话上下文（内存存储即可，无需数据库）
12 - 端口 3000，前端静态文件通过 Express 托管
13 【项目结构】
14 - 创建 package.json，包含所需依赖
15 - 创建 server.js（后端入口）
16 - 将前端 HTML 放在 public/ 目录下
17 - 创建 .env.example 文件说明需要配置的环境变量
18 - 创建启动脚本
```

## 4.2 观察 Claude Code 的执行过程

发送上述 Prompt 后，Claude Code 会展现出 Agent 的完整工作流。以下是你在终端中会看到的典型执行序列：

| 开发流程与工具调用计划<br>(DEVELOPMENT PROCESS & TOOL CALL PLAN) |         |                                                                                                                   |
|-------------------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------|
| 步骤                                                    | 工具调用    | 具体操作                                                                                                              |
| 1                                                     | 💡 Think | 分析需求, 规划项目架构和文件结构                                                                                                 |
| 2                                                     | ⚙️ Bash | <code>mkdir -p public</code> 创建目录结构                                                                               |
| 3                                                     | 📝 Write | 创建 <code>package.json</code> (定义依赖: <code>express</code> 、 <code>@anthropic-ai/sdk</code> 、 <code>dotenv</code> ) |
| 4                                                     | 📝 Write | 创建 <code>server.js</code> (Express 服务器 + Anthropic API 调用 + SSE 流式响应)                                             |
| 5                                                     | 📖 Read  | 读取现有的 <code>chatbot.html</code> , 分析其结构                                                                           |
| 6                                                     | 📝 Write | 创建 <code>public/index.html</code> (基于原始设计, 改造为调用后端 API)                                                           |
| 7                                                     | 📝 Write | 创建 <code>.env.example</code> (说明环境变量配置)                                                                           |
| 8                                                     | ⚙️ Bash | <code>npm install</code> 安装依赖                                                                                     |
| 9                                                     | ⚙️ Bash | <code>node server.js</code> 启动服务并测试                                                                               |

整个过程约 30-60 秒，Claude Code 会自主完成从规划到部署的全部步骤。

4.3 运行验证

Claude Code 完成后，你需要配置 API Key 才能让后端正常调用大模型：

代码块

```
1  # 复制环境变量模板
2  cp .env.example .env
3  # 编辑 .env 文件，填入你的 API Key
4  # DEEPSEEK_API_KEY=sk-your-key-here
```

然后启动应用：

代码块

```
1  npm start
```

在浏览器中打开 <http://localhost:3000>，输入一条消息——如果一切配置正确，你会看到 AI 的回复以流式方式逐字出现。恭喜，你刚刚用 Claude Code 在 60 秒内生成了一个完整的全栈 ChatBot 应用。

4.4 分析生成的代码结构

```
chatbot-project/
├── package.json # 项目依赖 (express, @anthropic-ai/sdk, dotenv)
├── server.js # 后端入口 (Express + Anthropic API + SSE)
├── .env.example # 环境变量模板
├── public/
│   └── index.html # 前端页面 (基于 chatbot.html 改造)
├── chatbot.html # 原始前端原型 (保留参考)
└── node_modules/ # 依赖包
```

关键代码亮点（值得在 server.js 中关注）：

| 技术点及实现方式 (Technical Points & Implementation)                                                                                                                             |                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| 技术点                                                                                                                                                                      | 实现方式                                                                                                                             |
|  流式响应  | <code>res.setHeader('Content-Type', 'text/event-stream') + Anthropic SDK 的 <code>stream: true</code></code>                      |
| 多轮对话                                                                                                                                                                     | 前端维护 <code>messages</code> 数组, 每次请求携带完整历史     |
| 错误处理                                                                                                                                                                     | <code>try-catch</code> 包裹 API 调用, 返回结构化错误信息  |
| CORS                                                                                                                                                                     | Express 内置静态文件托管, 无跨域问题                     |

4.5 不满意？Checkpoint 回滚

如果对 Claude Code 生成的代码结构或实现方式不满意，不需要手动删除文件重来。还记得前面课程中介绍的 Checkpoint 机制吗？

```
代码块
1  /rewind
```

选择生成代码之前的 Checkpoint 节点，所有新创建的文件会被删除，修改过的文件恢复原状——干净回到起点，然后调整 Prompt 重新生成。

5、代码开发（方案二）：多 Agent 流水线协作

方案一的"一句话全自动"虽然快，但本质上是让一个 Agent 从头到尾包揽所有工作——需求分析、架构设计、编码实现、质量审查全部由同一个上下文完成。在简单项目中这没有问题，但在工业级开发中，这种"独狼模式"有明显的局限性：

- 1 请在 `.claude/agents/` 目录下创建以下三个 Agent 配置文件:
- 2 [1. chatbot-planner.md](#) — 产品架构师

```
3  - 模型: sonnet
4  - 只读工具: Read, Glob, Grep, WebSearch
5  - 权限模式: plan
6  - 最大轮次: 10
7  - Prompt: 你是一位前端产品架构师, 负责分析需求、设计页面结构、
8  规划交互流程和技术方案, 输出实现计划。你只做规划, 不写代码。
9  2. chatbot-coder.md — 前后端开发工程师
10 - 模型: sonnet
11 - 读写工具: Read, Write, Edit, Bash, Glob, Grep
12 - 权限模式: acceptEdits
13 - 最大轮次: 20
14 - Prompt: 你是一位前后端开发工程师, 根据规划方案编写完整的代码实现。
15 使用现代 CSS 和 ES6+ JavaScript, 注重代码质量和用户体验。
16 3. chatbot-reviewer.md — 代码审查专家
17 - 模型: haiku
18 - 只读工具: Read, Glob, Grep
19 - 后台运行: true
20 - 最大轮次: 10
21 - Prompt: 你是一位代码审查专家, 重点检查可访问性、性能、安全性、
22 代码质量和响应式设计, 输出评分和改进建议。
```

Claude Code 会创建 `.claude/agents/` 目录并生成三个 `.md` 文件。以 `chatbot-planner.md` 为例, 生成的文件内容大致如下:

代码块

```
1  ---
2  name: chatbot-planner
3  description: 规划 HTML 页面结构和交互方案
4  model: sonnet
5  maxTurns: 10
6  allowedTools:
7  - Read
8  - Glob
9  - Grep
10 - WebSearch
11 permissionMode: plan
12 ---
13 你是一位前端产品架构师。你的职责是:
14 1. 分析用户需求, 规划页面结构
15 2. 设计交互流程和 UI 布局
16 3. 列出技术方案 (CSS 框架、动画、响应式策略)
17 4. 输出一份简洁的实现计划 (不要写代码)
18 输出格式:
19 ## 需求分析
20 ## 页面结构设计
```

```
21  ## 交互流程
22  ## 技术方案
23  ## 实现步骤清单
```

创建完成后，验证文件结构：

代码块

```
1  ls .claude/agents/
2  # 应输出: chatbot-coder.md chatbot-planner.md chatbot-reviewer.md
```

## 5.3 执行流水线

三个 Agent 就位后，按照**规划** → **编码** → **审查**的顺序逐步执行。

### Step 1: 启动 Planner Agent

在 Claude Code 中输入：

代码块

```
1  @chatbot-planner 请为以下需求规划一个完整的前后端 ChatBot 应用：
2  基于当前目录的 chatbot.html 前端设计，开发一个可运行的 ChatBot，
3  前端通过 fetch 调用后端 API，后端使用 Node.js + Express + DeepSeek(我的key是：sk-
   553c1c39b02f40c193a2b0e4b8d41b3c)
4  支持流式输出和多轮对话。
```

@agent名称 是调用自定义 Agent 的语法。Planner 会分析需求，输出一份结构化的实现计划——包含需求分析、页面结构、技术方案和实现步骤清单。注意：由于 Planner 只有Read/Glob/Grep 等只读工具，**它不会也不能修改任何文件**。

### Step 2: 启动 Coder Agent

Planner 输出计划后，将计划交给 Coder：

代码块

```
1  @chatbot-coder 根据刚才 chatbot-planner 给出的规划方案，
2  实现完整的前后端 ChatBot 应用。前端保存在 public/index.html，
3  后端入口为 server.js，包含 package.json 和 .env.example。
```

Coder 拥有 Write/Edit/Bash 等读写工具，会按照 Planner 的方案逐步创建文件、编写代码、安装依赖。

### Step 3: 启动 Reviewer Agent（后台运行）

编码完成后，启动 Reviewer 进行代码审查：

代码块

1 @chatbot-reviewer 请审查 server.js 和 public/index.html 的代码质量

由于 chatbot-reviewer 配置了 background: true ，它会进入**后台运行**——不阻塞你的主对话。你可以在等待审查结果的同时继续做其他事情。审查完成后，会收到一份包含评分和改进建议的报告。

5.4 方案一 vs 方案二对比

| 维度   | 方案一（一句话生成）                                                                                   | 方案二（多 Agent 流水线）                                                                                                                                                                                                                                                                 |
|------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 适用场景 | 快速原型、小型项目、个人开发                                                                               | 复杂项目、团队协作，工业级开发                                                                                                                                                                                                                                                                  |
| 代码质量 | 依赖单次生成质量，无内置审查                                                                               | 分阶段审查（Reviewer 自动检查），质量更可控                                                                                                                                                                                                                                                       |
| 可定制性 | 低——需要在 Prompt 中描述所有要求                                                                        | 高——每个 Agent 独立配置模型，工具权限，提示词                                                                                                                                                                                                                                                      |
| 可复用性 | 低——每次都要重新描述需求                                                                                | 高——Agent 和工作流可跨项目复用                                                                                                                                                                                                                                                              |
| 成本控制 | 全程使用同一模型  | 精细控制——规划和编码用 Sonnet，审查用 Haiku 降本  计划代码 ← 高 → 审查                                                                                                                                             |
| 上手难度 | 零门槛                                                                                          | 需要理解 Agent 体系和工作流概念                                                                                                                                                                                                                                                              |
| 执行耗时 | ~60 秒     | ~3-5 分钟（但质量更高）    |

**核心认知：**方案一和方案二并不是"非此即彼"的关系。在实际开发中，最高效的做法是**组合使用**——用方案一快速生成初版原型，确认方向正确后，用方案二的 Reviewer Agent 进行质量审查，再根据审查建议迭代优化。

6、本节小结

- 1. Claude Code 不只是"帮你写代码"的工具——它是一个可编排的 Agent 系统，能模拟团队协作流程；
- 2. Prompt 质量决定输出质量——同样的功能，精心设计的 Prompt 和随意描述的结果天壤之别；

3. **设计和开发可以无缝衔接**——通过 /frontend-design 和 Pencil MCP，从视觉设计到代码实现不再割裂；
4. **Checkpoint 让试错成本归零**——大胆尝试任何方案，不满意就一键回滚。